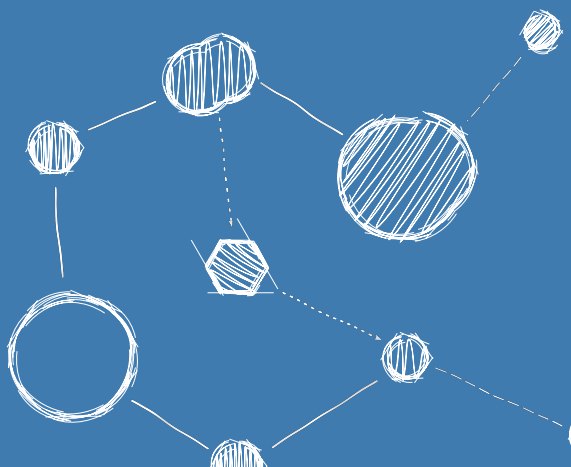
institut für astrophysik
UNIVERSITÄTSSTERNWARTE WIEN

universität wien

FLIGHTOS – AN RTOS FOR SPACE-BASED ASTRONOMY

Software Requirements Specification



Software Requirements Specification

Reference: FLIGHTOS-UVIE-SRS-001

Version: Issue 1.1, August 31, 2026

Prepared by: Armin Luntzer[1]

Checked by: Roland Ottensamer[1]

Approved by: Franz Kerschbaum[1]

Authorised by: -

[1] Department of Astrophysics, University of Vienna

Contents

List of Requirements

Revision History

Revision	Date	Author(s)	Description
0.0	24.07.2015	AL	draft requirements created based on NGAPP
0.1	30.03.2016	AL	initial version with specifications from MPPBv2
0.2	11.04.2016	AL	revised after internal design review
1.0	01.06.2017	FK	document approved
1.1	31.08.2026	AL	adopt FlightOS name, add refernces to LEON3/4 targets, update requirements

1. Introduction

1.1 Purpose of the Document

This document specifies the software requirements for the operating system *FlightOS*. FlightOS targets LEON3-based platforms such as *GR712RC* and *GR740*. It also has legacy support for the Scalable Sensor Data Processor (SSDP), and to a lesser extent, its compatible predecessor, the MPPB v2.x [1]. It is intended to be used in unmanned space applications of at least Software Criticality Level C. Readers must be familiar with the basic concepts of event driven real time operating systems and the target hardware.

This document follows the document structure for software requirement specifications found in Annex D of ECSS-E-ST-40C[2].

1.2 Scope of the Software

FlightOS is a lightweight operating system for space-based astronomy applications for European SPARC-based processors.

2. Applicable and Reference Documents

[1] *Massively Parallel Processor Breadboarding Datasheet*. 2016.

[2] *ECSS-E-ST-40C Space engineering - Software*. ESA Requirements and Standards Division, 2009.

[3] *FlightOS Architectural Design Document*. 2026.

[4] *FlightOS Test Specification*. 2026.

3. Terms, Definitions and Abbreviated Items

3.1 Acronyms

ADC	Analog to Digital Converter
API	Application Programming Interface
BSP	Board Support Package
CPU	Central Processing Unit
DAC	Digital to Analog Converter
DMA	Direct Memory Access
DSP	Digital Signal Processor
ELF	Executable and Linkable Format
FIFO	First In - First Out
FPU	Floating Point Unit
GCC	GNU Compiler Collection
ILP	Instruction Level Parallelism
ISR	Interrupt Service Routine
MMU	Memory Management Unit
MPPB	Massively Parallel Processor Breadboarding system
NGAPP	Next Generation Astronomy Processing Platform
NoC	Network On Chip
POSIX	Portable Operating System Interface
PUS	Packet Utilisation Standard
RAM	Random-Access Memory
RISC	Reduced Instruction Set Computing
RMAP	Remote Memory Access Protocol
RR	Round Robin
RSA	RUAG Space Austria 8
SMP	Symmetric Multiprocessing
SoC	System On Chip
SSDP	Scalable Sensor Data Processor
TCM	Tightly-Coupled Memory
UVIE	University of Vienna 8
VLIW	Very Long Instruction Word

3.2 Glossary

LEON2

The LEON2 is a synthesisable VHDL model of a 32-bit processor compliant with the SPARC V8 architecture. It is highly configurable and particularly suitable for System On Chip (SoC) designs. Its source code is available under the GNU LGPL license

LEON3

The LEON3 is an updated version of the LEON2, changes include Symmetric Multiprocessing (Symmetric Multiprocessing (SMP)) support and a deeper instruction pipeline

LEON3-FT

The LEON3-FT is a fault-tolerant version of the LEON3. Changes to the base version include autonomous error handling, cache locking and different cache replacement strategies.

SPARC

SPARC (”scalable processor architecture”) is a Reduced Instruction Set Computing (Reduced Instruction Set Computing (RISC)) instruction set architecture developed by Sun Microsystems in the 1980s. The distinct feature of SPARC processors is the high number of Central Processing Unit (Central Processing Unit (CPU)) registers that are accessed similarly to stack variables via “sliding windows”.

Xentium

The Xentium is a high performance Very Long Instruction Word (Very Long Instruction Word (VLIW)) Digital Signal Processor (DSP) core. It operates 10 parallel execution slots supporting 32/40 bit scalar and two 16-bit element vector operations.

4. Software Overview

4.1 Function and Purpose

In the course of the NGAPP activities, an evaluation of the MPPB was performed in a joint effort of RUAG Space Austria (RSA) and the Department of Astrophysics of the University of Vienna (UVIE). While the original intent of the work of UVIE was to quantify the performance of the Xentium DSPs and the MPPB as a whole with regard to on-board data treatment and reduction in an astronomical mission setting, it was found that, given the highly innovative nature of this new processing platform, a novel approach was needed concerning the management of system resources, Direct Memory Access (DMA) mechanics and DSP program design for best efficiency and turnover rates. Consequently, UVIE developed an experimental operating system to stably drive the DSP cores and the MPPB close to its performance limit. FlightOS is a development based on this operating system concept. After the SSDP project was abandoned, the operating system was generalised to support the LEON3 family of European processors for space. In addition, the project also provides software test coverage, example applications and documentation that supports all aspects of the software.

4.2 Environmental considerations

FlightOS currently primarily supports the *GR712RC* and *GR740*, but legacy functionality to operate the SSDP and MPPB v2.x is available. Ultimately, this does allow reuse of the operating system core for LEON2 and LEON3 based platforms with minor adaptations. For multi-core processors SMP support is available. The operating system can also make use of the SPARC SRMMU.

FlightOS is released under an open source license, so compatible development tools should be available under such licenses as well. Note that it might still be necessary to use a commercial product to target particular configurations or proprietary hardware functionality.

4.3 Relation to other systems

FlightOS is a stand-alone software product.

4.4 Constraints

FlightOS primarily targets the *GR712RC* and *GR740* processors, which effectively limits the hardware support to the characteristics of this platform.
The source code is written in C and if required, SPARCv8 compatible assembly. This is necessary as hardware interaction at a machine level requires programming languages designed for that purpose.

5. Requirements

5.1 General

R-GEN-0001	Short Text	Software Requirement
	No external dependencies	FlightOS shall not make use of external libraries, including C-runtime libraries.

R-GEN-0002	Short Text	Software Requirement
	Custom libc functionality	FlightOS shall, as part of its code-base and as needed, provide its own implementations of typical C-libary functions with an API conforming to their POSIX definitions.
Comment	The collection of these functions can also form the base or a subset of a C libary to be used by applications.	

R-GEN-0003	Short Text	Software Requirement
	Application libc usage	FlightOS shall optionally call a function that initialises a C-library on startup. It is up to the user to define and link the implementation of this function and the according C-library.
Comment	The function __libc_start_main() is declared as a weak symbol and tested before execution.	

R-GEN-0004	Short Text	Software Requirement
	Fatal Error Management	If a non-recoverable error state is detected in FlightOS, it shall optionally call an error handling routine provided by the user before issuing a reboot. FlightOS shall provide a directive to register such a function.
Comment	This gives the user the possiblity to perform a clean shutdown of critical tasks in their particular environment.	

R-GEN-0006	Short Text	Software Requirement
	Core runtime	In its basic configuration, FlightOS shall restrict itself to the initialisation of its core services on a single processor, thereby configuring traps, memories and timers. All other services or device drivers shall be configured and added via the build system.

R-GEN-0601	Short Text	Software Requirement
	Error Logging	FlightOS shall maintain an error message log that is readable by the user.

R-GEN-0602	Short Text	Software Requirement
	Coding Style	FlightOS code shall use the Linux kernel coding style. Source files and patches shall be checked using the checkpatch.pl utility found in the Linux kernel source tree.
Comment	A major point of FlightOS is that it does not only come with an open source license, but should ideally only use tools that are distributable under a similar license. The Linux kernel is used in billions of devices word-wide, its coding style is hence arguably well-suited for use in successful software.	

5.2 Functional Requirements

R-FUN-0007	Short Text	Software Requirement
	SMP Support	FlightOS shall be able to run on a multi-core configuration of a LEON3 processor.

R-FUN-0803	Short Text	Software Requirement
	MMU Support	FlightOS shall support the use of the MMU of the LEON3 processor.

R-FUN-0008	Short Text	Software Requirement
	Supervisor Mode	The FlightOS kernel shall execute with the SPARC supervisor mode enabled. Application code shall run with supervisor mode disabled.

R-FUN-0011	Short Text	Software Requirement
	Timing	FlightOS shall provide access to typical time keeping, time taking and delay timing functionality expected by an operating system.

R-FUN-0012	Short Text	Software Requirement
	Task Support	FlightOS shall provide means to create new tasks.

R-FUN-0013	Short Text	Software Requirement
	Task priorities and deadlines	FlightOS shall support the assignment of a priority and a deadline to a task.

R-FUN-0014	Short Text	Software Requirement
	Task scheduling	FlightOS shall offer an Earliest Deadline First scheduler supporting priority execution in overload conditions.
Comment	Along with dynamic ticking, this is an option to optimise thread CPU utilisation with the added benefit of predictable execution for certain high-priority threads in conditions where the total load unexpectedly would exceed 100%.	

G-FUN-0015	Short Text	Software Requirement
	Other schedulers	FlightOS shall offer a fixed priority scheduler with priority inversion handling.

R-FUN-0016	Short Text	Software Requirement
	Kernel Ticks	FlightOS shall operate in tickless (non-periodic) timed wakeup mode by default.
Comment	Tickless timing avoids unnecessary wake-ups of the CPU if no task is running and improves performance by only switching to kernel mode from regular tasks when absolutely necessary.	

R-FUN-0017	Short Text	Software Requirement
	Tickless Timing	FlightOS timing functionality shall be able to operate in tickless mode, where a queue of wakeup times is maintained and a hardware timer is used in such a way that its next underflow (resulting in an interrupt) is programmed to coincide with the next wakeup time. New wakeup times shall be inserted into the queue as needed to maintain the desired timeline of events and the hardware timer be readjusted accordingly.

G-FUN-0019	Short Text	Software Requirement
	Semaphores and Mutexes	FlightOS should provide semaphores and mutexes as part of its task functionality. Task priorities shall be protected by the priority ceiling protocol.

R-FUN-0020	Short Text	Software Requirement
	Tasks and SMP	FlightOS shall support task migration between CPUs and track and ensure atomicity of related functions (e.g. mutexes) across multiple processors.

R-FUN-0021	Short Text	Software Requirement
	Message Queues	FlightOS shall support message queues for inter-process communication. Atomicity of queues shall be ensured across multiple processors.

R-FUN-0022	Short Text	Software Requirement
	Kernel Modules	FlightOS shall offer loadable module support infrastructure.

R-FUN-0023	Short Text	Software Requirement
	Application Loading	FlightOS shall be capable of parsing and executing applications in ELF format.

R-FUN-0804	Short Text	Software Requirement
	Kernel-Userspace Initialisation	FlightOS shall offer a configurable initialisation point that executes a user-space setup procedure.

Comment	This is the point where a project configuration can be initialised and the application software can be loaded.

R-FUN-0023	Short Text	Software Requirement
	System Control Interface	FlightOS shall offer a way for drivers or other functional modules to export configuration or state variables into a logical tree maintained by the operating system. This logical tree shall be accessible by other modules and the user as a character-based interface.
Comment	This is supposed to be a centrally organised, generic parseable interface to get or set configuration states or system statistics.	

R-FUN-0024	Short Text	Software Requirement
	Binary System Control Interface	FlightOS shall offer the possibility to install binary exchange nodes within the System Control Interface tree that may be used for larger amouts of binary data. The binary format shall be defined by the creator of the node. Any users of the node shall be responsible to read or write in the correct format expected by the node.
Comment	Sometimes, a binary dump to or from a subsystem is needed that is only parseable with special knowledge. This could, for example, be an internal memory dump or some calibration data.	

G-FUN-0025	Short Text	Software Requirement
	SSDP Device Drivers	FlightOS should come with device drivers for all hardware functionality of the SSDP.

R-FUN-0026	Short Text	Software Requirement
	Xentium Scheduler	FlightOS shall offer a way to define and load Xentium data processing code.

R-FUN-0027	Short Text	Software Requirement
	Xentium Data Buffers	FlightOS shall provide packet-driven meta-data buffers to Xentium programs that can link to arbitrary data sets and routing tables that define the path of a data packet through a series of Xentium program nodes.
Comment	Datagrams propagate through processing nodes as defined in their routing table, which represents their individual “processing chain”.	

R-FUN-0028	Short Text	Software Requirement
	GRSPW2 Device Drivers	FlightOS shall provide support for the GRSPW2 IP core.

R-FUN-0029	Short Text	Software Requirement
	SPI controller infrastructure	FlightOS shall provide infrastructure for SPI controller drivers.

R-FUN-0030	Short Text	Software Requirement
	GRSPI controller driver	FlightOS shall provide a driver for the GRSPI IP core.

5.3 Performance Requirements

R-GEN-0009	Short Text	Software Requirement
	Traps/Interrupt	FlightOS trap entry and exit code shall not exceed 500 instructions.
Comment	This does not include the callback functions for a trap or interrupt service routine.	

R-GEN-0018	Short Text	Software Requirement
	Deferred saving of FPU registers	In trap mode, FlightOS shall defer saving of FPU registers until it is accessed.
Comment	This approach saves many clock cycles, as the FPU is not typically used as part of an ISR.	

R-GEN-0010	Short Text	Software Requirement
	Interrupt downtime	ISRs that execute longer than 50 µs shall be set to be executed in deferred mode at a later time if feasible given the type and rate of an ISR.
Comment	Interrupt mode should be left as fast as possible, so regular processing can continue. Most ISRs requiring long processing times perform actions on data, which can typically be moved into a dedicated thread, with the ISR acting as a signalling function.	

R-GEN-0028	Short Text	Software Requirement
	Real-Time Thread Support	FlightOS shall offer support for a class of real time threads that may also preempt the operating sytem with the exception of ISRs.
Comment	Some application real time tasks may be so timing sensitive, that even operating system code must be preemptible to guarantee a timely response.	

5.4 Interface Requirements

R-GEN-2001	Short Text	Software Requirement
	Interface Documentation	The internal and external interfaces of FlightOS shall be described as part of its source code in Doxygen markup.
Comment	The interfaces of a complex piece of software such as an operating system often change over time, as it is adapted to new circumstances or improved implementation of particular functionality or just subtle changes that may not always propagate into other documents as they should. It is therefore better to maintain interface documentation together with the code it describes, where it will be more likely to be updated on the spot.	

5.5 Operational Requirements

No operational requirements have been identified.

5.6 Resources Requirements

R-GEN-0101	Short Text	Software Requirement
	SSDP Target Platform	FlightOS shall support on the SSDP, in particular the LEON3-FT processor of the platform. It should also support the predecessor MPPB v2.x.

R-GEN-0102	Short Text	Software Requirement
	GR712RC Target Platform	FlightOS shall support on the *GR712RC*.

R-GEN-0103	Short Text	Software Requirement
	GR740 Target Platform	FlightOS shall support on the *GR740*.

5.7 Design Requirements and Implementation Constraints

R-GEN-0801	Short Text	Software Requirement
	Modular Design	All components of FlightOS shall be written such that a particular component does contain its intended functionality as much as possible.
Comment	If something is mostly self-contained, it is easier to modify and re-use in another software project.	

R-GEN-0802	Short Text	Software Requirement
	Software Hierarchy	All components shall make use and rely only on functionality that is lower in hierarchy. The use of functionality that is hierarchically equal or higher shall be explicitly forbidden.
Comment	Note that hierarchy refers to the abstraction level of a component. An ISR is of higher level than the interrupt dispatcher. Even though it is called by the latter, the actual registration of the ISR is done by a higher level component. Such constructs are legal, the reliance on user-space provided functionality, e.g. an error reporting function, which sends messages via some packet interface on the other hand, is not.	

R-GEN-0805	Short Text	Software Requirement
	Programming Language	The programming language shall be C. SPARCv8 compatible assembly shall be used when necessitated by performance or timing constraints and interface requirements.

5.8 Security and Privacy Requirements

No security or privacy requirements have been identified.

5.9 Software Quality Requirements

G-GEN-0201	Short Text	Software Requirement
	Product Metrics	FlightOS should have a cyclomatic complexity of at most 15 and a nesting level of at most 6 per function. Each function shall have a single exit point for the nominal case.

5.10 Software Reliability Requirements

No software reliability requirements have been identified.

5.11 Software Maintainability Requirements

R-GEN-0301	Short Text	Software Requirement
	Version Identification	It shall be possible to identifiy the version of compiled binary software components by reading their identifier from a special memory segment.
Comment	The build identifier or version number should be set and defined when building the binary.	

5.12 Software Safety Requirements

R-GEN-0401	Short Text	Software Requirement
	Stack Pointer Checks	The stack pointer of a task shall be checked for feasibility before scheduling the latter.

R-GEN-0402	Short Text	Software Requirement
	Correctable Traps	FlightOS shall provide handlers for correctable traps caused by kernel or user code and either correctly execute the desired operation (e.g. unaligned access) or replace the result with a default value (e.g. divison by zero) and skip the offending code instruction to continue.

R-GEN-0403	Short Text	Software Requirement
	Trap Error Reporting	FlightOS shall make an entry into its error message log when a trap event occurs, describing the nature and source of the trap.

5.13 Software Configuration and Delivery Requirements

R-GEN-0005	Short Text	Software Requirement
	Build configuration	FlightOS shall make use of the Linux Kernel Build System (kbuild) for its configuration.

5.14 Data Definition and Database Requirements

No data definition or database requirements have been identified.

5.15 Human Factors Related Requirements

No human factors related requirements have been identified.

5.16 Adaptation and Installation Requirements

No adaptation and installation requirements have been identified.

6. Validation Requirements

R-GEN-1001	Short Text	Software Requirement
	Verification Method	The verification method and verification activity shall be specified in a Software Qualification Test Plan for each requirement.

R-GEN-1002	Short Text	Software Requirement
	Qualification Testing	SW qualification testing shall be performed with various configurations of FlightOS.

7. Traceability

The requirements in this document are both user and system requirements. Traces from design and test to the software requirements are given in the respective documents.[3][4]

8. Logical Model Description

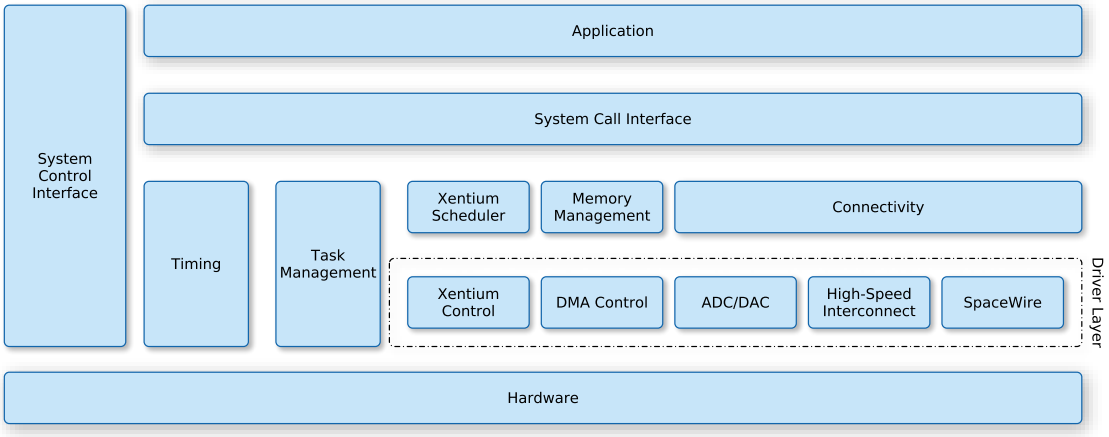


Figure 8.1: The logical model of FlightOS. Here, the ”hardware” layer represents both the hardware and the hardware abstraction layer of the software.

In FlightOS, hardware is accessed in multiple layers of abstraction (see Figure 8.1). Typical CPU tasks such as thread/task management and timer operation are used as part of the operating system kernel and are also accessible by user applications via a system call interface. Other functional hardware components of the SSDP such as the Network On Chip (NoC) DMA have their own driver modules. These are in turn used by the Xentium scheduler and other higher level modules in the operating system. Configuration of and access to the latter from user space is done via a system call interface. The system control interface serves as an intermediate between all layers of the operating system, where system or module states and hardware modes or usage statistics may be exported by individual components for external (user) access.